Temat: Od Dockera do Kubernetesa · dzień 3: stan, zadania, zasoby, planowanie

Termin: 5 dni, 9:00–17:00

Prowadzi: Dawid Marniok

Copyright (C) 2026 Sages sp. z o.o. Kopiowanie, przetwarzanie, rozpowszechnianie tych materiałów w całości lub w części jest zabronione.

Dzień 3

Stan, zadania, zasoby i planowanie podów

sages

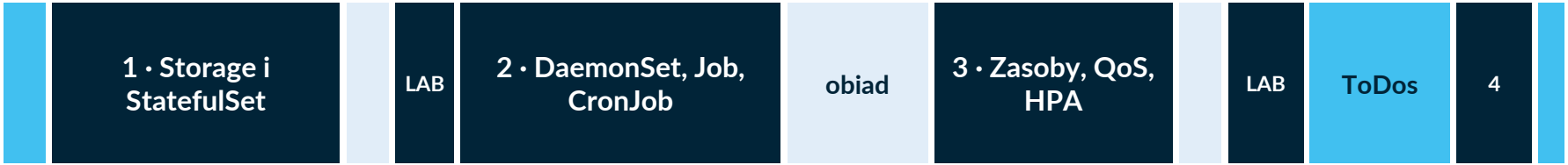
Powtórka dnia 2: trzy pytania na czat

1	Usuwasz ręcznie pod z Deploymentu. Co się stanie?	Deployment · ReplicaSet
2	Skąd Service wie, do których podów wysłać ruch?	Service · EndpointSlice
3	Zmieniasz ConfigMapę. Czy pod z env widzi nową wartość?	ConfigMap · env vs wolumen

na czat: numer pytania i jedno zdanie · odpowiedzi omawiam po minucie

Copyright (C) 2026 Sages sp. z o.o. Kopiowanie, przetwarzanie, rozpowszechnianie tych materiałów w całości lub w części jest zabronione.

Plan dnia: cztery tematy, trzy laby i ToDos



dane
przeżywają poda

zadania
mają koniec

zasoby
mają granice

pody
mają adres węzła

laby: labs/lab09 … lab11 i labs/todos-dzien3 · gotowe manifesty: przyklady/k8s

Copyright (C) 2026 Sages sp. z o.o. Kopiowanie, przetwarzanie, rozpowszechnianie tych materiałów w całości lub w części jest zabronione.

sages

TERAZ · Na start

Sprawdź namespace, StorageClass i quotę

```
cd kubernetes-w-praktyce && git pull                  → labs/lab09 … todos-
dzien3
kubectl config get-contexts                           → kolumna NAMESPACE
kubectl get storageclass                              → która ma (default)?
kubectl describe resourcequota                        → ile PVC i CPU zostało
kubectl get limitrange                                → jest czy nie ma?
```

NA CZAT nazwa domyślnej StorageClass · limit PVC z quoty

Copyright (C) 2026 Sages sp. z o.o. Kopiowanie, przetwarzanie, rozpowszechnianie tych materiałów w całości lub w części jest zabronione.

sages

1

DZIEŃ 3 · TEMAT 1

Storage i StatefulSet

Po tym temacie dajesz aplikacji dysk, który przeżyje restart poda, i wiesz, kiedy potrzebny jest StatefulSet.

Copyright (C) 2026 Sages sp. z o.o. Kopiowanie, przetwarzanie, rozpowszechnianie tych materiałów w całości lub w części jest zabronione.

sages

Kontener jest ulotny, dane giną razem z nim

system plików kontenera znika przy każdym restarcie, a nowy pod startuje od zera

`emptyDir`

żyje tyle, co pod

cache, pliki tymczasowe, wymiana danych między kontenerami poda

`hostPath`

żyje tyle, co węzeł

tylko agenci systemowi (logi, metryki); dla aplikacji: nie

`PVC`

żyje niezależnie od poda

dane aplikacji: baza, uploady, kolejka na dysku

dane, które mają przetrwać, trzymasz poza podem: w wolumenie z PVC

Copyright (C) 2026 Sages sp. z o.o. Kopiowanie, przetwarzanie, rozpowszechnianie tych materiałów w całości lub w części jest zabronione.

sages

PVC zamawia, PV dostarcza, StorageClass tworzy



statycznie: admin najpierw, PVC szuka pasującego PV · dynamicznie: PVC uruchamia tworzenie dysku

Copyright (C) 2026 Sages sp. z o.o. Kopiowanie, przetwarzanie, rozpowszechnianie tych materiałów w całości lub w części jest zabronione.

sages

PVC to zamówienie na dysk, Pod tylko go montuje

PVC

```yaml
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: app-data
spec:
  accessModes: [ReadWriteOnce]
  storageClassName: gp3
  resources:
    requests:
      storage: 1Gi
```

POD (FRAGMENT)

```yaml
spec:                          # Pod
  containers:
  - name: web
    image: nginx:1.29
    volumeMounts:
    - name: data
      mountPath: /usr/share/nginx/html
  volumes:
  - name: data
    persistentVolumeClaim:
      claimName: app-data
```

kubectl get pvc → Bound = gotowe · Pending = czeka na poda albo nie ma pasującego PV

Copyright (C) 2026 Sages sp. z o.o. Kopiowanie, przetwarzanie, rozpowszechnianie tych materiałów w całości lub w części jest zabronione.

sages

RWO znaczy jeden węzeł, nie jeden pod

Tryb	Co znaczy	Na EKS
RWO · ReadWriteOnce	zapis z jednego węzła	EBS · gp3
RWOP · ReadWriteOncePod	zapis z jednego poda (GA 1.29)	EBS przez CSI
ROX · ReadOnlyMany	odczyt z wielu węzłów	EFS
RWX · ReadWriteMany	zapis z wielu węzłów	EFS · efs.csi.aws.com

`reclaimPolicy: Delete`
usuwasz PVC → znika PV i dysk (domyślne)

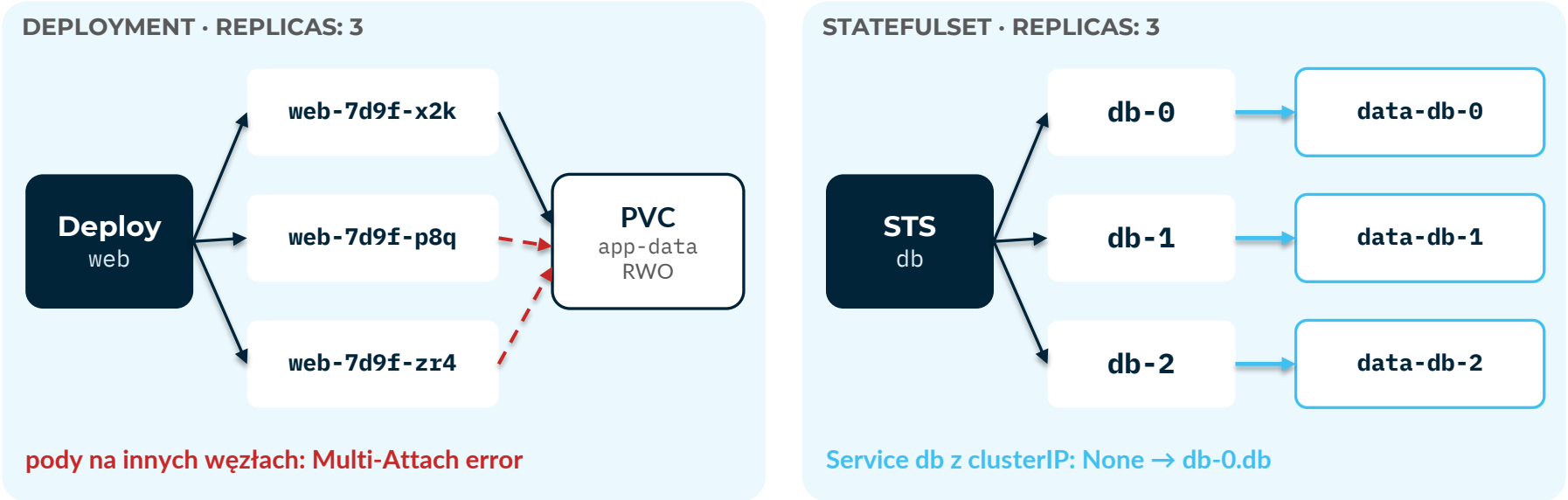
`reclaimPolicy: Retain`
PV zostaje jako Released, dane sprzątasz ręcznie

dziś tylko CSI: wbudowane provisionery (aws-ebs, azure-disk, gce-pd) usunięto z Kubernetesa

Copyright (C) 2026 Sages sp. z o.o. Kopiowanie, przetwarzanie, rozpowszechnianie tych materiałów w całości lub w części jest zabronione.

sages

StatefulSet: każdy pod ma imię i własny dysk



Deployment: pody wymienne, wspólny dysk · StatefulSet: stała nazwa, kolejność, PVC per pod

Copyright (C) 2026 Sages sp. z o.o. Kopiowanie, przetwarzanie, rozpowszechnianie tych materiałów w całości lub w części jest zabronione.

StatefulSet: serviceName i volumeClaimTemplates

```
spec:                    # StatefulSet db
  serviceName: db        # headless Service
  replicas: 3
  selector: {matchLabels: {app: db}}
  template:
    metadata: {labels: {app: db}}
    spec:
      containers:
      - name: db
        image: nginx:1.29
        volumeMounts: [{name: data, mountPath: /data}
]
  volumeClaimTemplates:
  - metadata: {name: data}
    spec:
      accessModes: [ReadWriteOnce]
      resources: {requests: {storage: 1Gi}}
```

Service db, clusterIP: None
tworzysz sam, przed StatefulSetem

PVC: data-db-0 … data-db-2
powstają przy pierwszym starcie poda

start 0 → 1 → 2
skalowanie w dół od najwyższego numeru

rolling update od db-2 do db-0
partition: aktualizuj tylko część

Copyright (C) 2026 Sages sp. z o.o. Kopiowanie, przetwarzanie, rozpowszechnianie tych materiałów w całości lub w części jest zabronione.

Usuwasz StatefulSet, dyski zostają

PVC przeżywają StatefulSet	kubectl delete sts db → data-db-* zostają i kosztują	**persistentVolumeClaimRetentionPolicy (GA 1.32)**
Multi-Attach error	RWO w Deploymencie z kilkoma replikami na różnych węzłach	**StatefulSet albo RWX, jeśli aplikacja to wytrzyma**
exceeded quota	u nas limit 6 PVC na namespace, STS z 3 replikami szybko go zjada	**kubectl describe quota, sprzątaj PVC po ćwiczeniu**

Copyright (C) 2026 Sages sp. z o.o. Kopiowanie, przetwarzanie, rozpowszechnianie tych materiałów w całości lub w części jest zabronione.

sages

Baza w Kubernetesie: z operatorem albo wcale

MA SENS, GDY

zespół zna i Kubernetesa, i tę bazę

operator robi backup, failover, upgrade

restore był przećwiczony

dev i test: baza na żądanie per gałąź

LEPIEJ USŁUGA ZARZĄDZANA, GDY

mały zespół bez dyżurów bazodanowych

„StatefulSet z obrazem postgres i tyle”

nikt nie wie, jak odtworzyć dane

chmura daje to samo za rozsądne pieniądze

StatefulSet to klocek, operator to administrator bazy w kodzie · operatory: dzień 5

Copyright (C) 2026 Sages sp. z o.o. Kopiowanie, przetwarzanie, rozpowszechnianie tych materiałów w całości lub w części jest zabronione.

sages

▶ DEMO · Dane przeżywają poda

Pod umiera, dysk i nazwa wracają

Cel: PVC i baza na dysku, potem StatefulSet z trzema replikami i PVC per pod.

OBSERWUJCIE

1. PVC Pending, potem Bound
2. pody 0 → 1 → 2 po kolei
3. web-1 wraca z tym samym PVC

Copyright (C) 2026 Sages sp. z o.o. Kopiowanie, przetwarzanie, rozpowszechnianie tych materiałów w całości lub w części jest zabronione.

sages

LAB `labs/lab09-storage-statefulset/`

Dysk i StatefulSet u siebie

1. PVC app-data i pod: plik przeżywa usunięcie poda
2. headless Service i StatefulSet db z dyskiem per pod
3. skalowanie 2 → 3 → 1: kolejność podów, co z PVC
4. sprzątanie PVC (quota), chętni: Multi-Attach, RWX

SUKCES plik w /data po delete pod, PVC data-db-0 i data-db-1 Bound

Copyright (C) 2026 Sages sp. z o.o. Kopiowanie, przetwarzanie, rozpowszechnianie tych materiałów w całości lub w części jest zabronione.

sages

Postawilibyście produkcyjną bazę na samym StatefulSecie?

Następny temat: DaemonSet, Job i CronJob

pod na każdym węźle, zadanie do skutku i zadanie według zegara

Copyright (C) 2026 Sages sp. z o.o. Kopiowanie, przetwarzanie, rozpowszechnianie tych materiałów w całości lub w części jest zabronione.

sages

2

DZIEŃ 3 · TEMAT 2

DaemonSet, Job, CronJob

Po tym temacie uruchamiasz agenta na każdym węźle, zadanie do skutku i zadanie według harmonogramu.

Copyright (C) 2026 Sages sp. z o.o. Kopiowanie, przetwarzanie, rozpowszechnianie tych materiałów w całości lub w części jest zabronione.

sages

Nie każdy proces to serwer, który działa bez końca

DaemonSet

jeden pod na każdym węźle

agent logów i metryk, CNI, CSI, kube-proxy

Job

uruchom i doprowadź do końca

migracja bazy, import, generowanie raportu

CronJob

Job według harmonogramu

backup, czyszczenie, raport o 6:00

Deployment pilnuje liczby replik · DaemonSet pilnuje węzłów · Job pilnuje ukończeń

zły kontroler = pętla restartów (Job jako Deployment) albo brak agenta na nowym węźle

Copyright (C) 2026 Sages sp. z o.o. Kopiowanie, przetwarzanie, rozpowszechnianie tych materiałów w całości lub w części jest zabronione.

DaemonSet idzie za węzłami, nie za liczbą replik

```
# DaemonSet log-agent: spec.template.spec
tolerations:
- key: node-role.kubernetes.io/control-plane
  operator: Exists
  effect: NoSchedule
containers:
- name: agent
  image: busybox:1.37
  command: ["sh", "-c", "ls /var/log/pods; sleep 3600"]
  volumeMounts:
  - {name: pods, mountPath: /var/log/pods, readOnly: true}
volumes:
- {name: pods, hostPath: {path: /var/log/pods}}
```

nowy węzeł → nowy pod
bez skalowania i bez replik

control-plane ma taint
bez tolerancji agent go omija

logi: /var/log/pods
nie /var/lib/docker: na węzłach jest containerd

rollout jak w Deploymencie
maxUnavailable, węzeł po węźle

Copyright (C) 2026 Sages sp. z o.o. Kopiowanie, przetwarzanie, rozpowszechnianie tych materiałów w całości lub w części jest zabronione.

sages

Job kończy się sukcesem albo limitem prób

```
# Job resize (fragment)
spec:
  completions: 5
  parallelism: 2
  completionMode: Indexed
  backoffLimit: 3
  activeDeadlineSeconds: 300
  ttlSecondsAfterFinished: 600
  template:
    spec:
      restartPolicy: Never
      containers:
      - name: worker
        image: busybox:1.37
        command: ["sh", "-
c", "echo $JOB_COMPLETION_INDEX"]
```

completions · parallelism
ile sukcesów łącznie, ile podów naraz

completionMode: Indexed
każdy pod dostaje numer 0…4

backoffLimit (domyślnie 6)
ile nieudanych podów do porażki Joba

activeDeadlineSeconds
twardy limit czasu całego Joba

ttlSecondsAfterFinished
po tylu sekundach Job i pody znikają

Copyright (C) 2026 Sages sp. z o.o. Kopiowanie, przetwarzanie, rozpowszechnianie tych materiałów w całości lub w części jest zabronione.

sages

CronJob tworzy Joby według zegara

```
spec:                          # CronJob backup
  schedule: "0 2 * * *"
  timeZone: Europe/Warsaw
  concurrencyPolicy: Forbid
  startingDeadlineSeconds: 300
  successfulJobsHistoryLimit: 3
  failedJobsHistoryLimit: 1
  jobTemplate:
    spec:
      template:
        spec:
          restartPolicy: OnFailure
          containers:
          - name: backup
            image: busybox:1.37
            command: ["date"]
```

Polityka	gdy poprzedni trwa
Allow (domyślnie)	startuje drugi obok
Forbid	nowy jest pomijany
Replace	stary usuwany, startuje nowy

schedule: min godz dzień mies dzień-tyg

`"*/5 * * * *"` co 5 minut

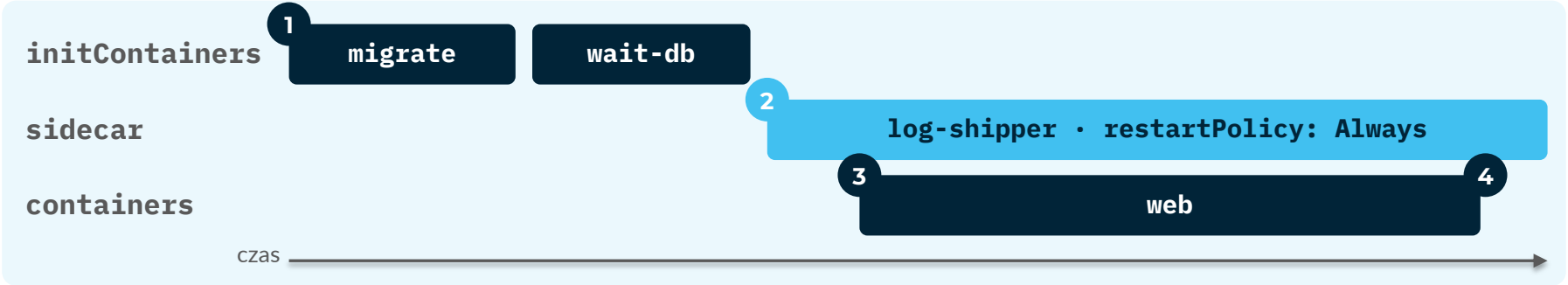
`"0 2 * * 1"` poniedziałek 2:00

bez timeZone liczy się strefa kube-controller-managera (zwykle UTC)

Copyright (C) 2026 Sages sp. z o.o. Kopiowanie, przetwarzanie, rozpowszechnianie tych materiałów w całości lub w części jest zabronione.

sages

Init najpierw, sidecar obok, aplikacja na końcu




```yaml
initContainers:
- name: migrate
  image: busybox:1.37
  command: ["sh", "-c", "echo migracja"]
- name: log-shipper
  image: busybox:1.37
  command: ["sh", "-c", "tail -F /logs/app.log"]
  restartPolicy: Always       # sidecar, 1.29+
```

Job + sidecar kończy się z aplikacją
wcześniej sidecar trzymał Job w nieskończoność

init pada → pod nie wystartuje
Init:CrashLoopBackOff, kubectl logs -c migrate

Copyright (C) 2026 Sages sp. z o.o. Kopiowanie, przetwarzanie, rozpowszechnianie tych materiałów w całości lub w części jest zabronione.

sages

Jeden znak psuje Job, sprzątanie trzeba zlecić

	ŹLE	DOBRZE
$(date) w apostrofach	`args: ["echo 'czas: $(date)'"]`	`args: ["echo czas: $(date)"]`
Joby i pody wiszą	`brak ttlSecondsAfterFinished`	`ttlSecondsAfterFinished: 600`
restartPolicy: Always	`API odrzuca Job`	`Never: nieudany pod zostaje do logs`

w apostrofach shell nie rozwija $(...) · kubectl get jobs po tygodniu pokaże, czy sprzątasz

Copyright (C) 2026 Sages sp. z o.o. Kopiowanie, przetwarzanie, rozpowszechnianie tych materiałów w całości lub w części jest zabronione.

sages

▶ DEMO · Job z indeksami i CronJob

Pięć części, dwie naraz, każda wie, którą wziąć

Cel: Job Indexed ze slajdu, potem CronJob co minutę: zadanie 90 s, Forbid.

OBSERWUJCIE

1. maks. 2 pody Running
2. numer części w logach
3. pominięte przebiegi CronJoba

Copyright (C) 2026 Sages sp. z o.o. Kopiowanie, przetwarzanie, rozpowszechnianie tych materiałów w całości lub w części jest zabronione.

sages

LAB `labs/lab10-job-cronjob-daemonset/`

Job, CronJob i DaemonSet u siebie

1. Joby: hello, completions, parallelism, deadline
2. napraw CronJob, który wypisuje „$(date)”
3. concurrencyPolicy: Forbid i JobAlreadyActive
4. DaemonSet: jeden pod na każdym węźle (-o wide)

SUKCES Joby Complete albo Failed z DeadlineExceeded, DaemonSet: DESIRED = węzły

Copyright (C) 2026 Sages sp. z o.o. Kopiowanie, przetwarzanie, rozpowszechnianie tych materiałów w całości lub w części jest zabronione.

sages

Co z Waszego crona przenieślibyście do CronJoba, a co nie?

Po obiedzie: zasoby, QoS i autoskalowanie

requests, limits, kto ginie pierwszy i jak HPA dokłada repliki

Copyright (C) 2026 Sages sp. z o.o. Kopiowanie, przetwarzanie, rozpowszechnianie tych materiałów w całości lub w części jest zabronione.

sages

3

DZIEŃ 3 · TEMAT 3

Zasoby, QoS, autoskalowanie

Po tym temacie ustawiasz requests i limits świadomie i wiesz, kiedy HPA doda repliki.

Copyright (C) 2026 Sages sp. z o.o. Kopiowanie, przetwarzanie, rozpowszechnianie tych materiałów w całości lub w części jest zabronione.

sages

requests rezerwują, limits przycinają

	CPU	Pamięć
requests	scheduler rezerwuje · gwarantowany udział	scheduler rezerwuje · odniesienie przy eviction
limits	throttling: wolniej, ale żyje	OOMKilled · exit 137 · restart
bez wartości	bierze wolne CPU z węzła	może wypchnąć sąsiadów z węzła

```
resources:
  requests: {cpu: 250m, memory: 256Mi}
  limits:   {cpu: 500m, memory: 256Mi}
```

1 CPU = 1000m = jeden rdzeń

pamięć w Mi i Gi, nie w M i G

limit pamięci = request: bez niespodzianek

Copyright (C) 2026 Sages sp. z o.o. Kopiowanie, przetwarzanie, rozpowszechnianie tych materiałów w całości lub w części jest zabronione.

sages

Klasa QoS decyduje, kto ginie pierwszy

QoS = Quality of Service. Klasę nadaje Kubernetes sam, z requests i limits kontenerów

Klasa	Kiedy	Przy braku pamięci na węźle
Guaranteed	requests = limits, CPU i pamięć, każdy kontener	ewikowany na końcu
Burstable	jest choć jeden request albo limit	w środku: kto najbardziej ponad requests
BestEffort	brak requests i limits	ewikowany pierwszy

```
kubectl get pod web -o jsonpath='{.status.qosClass}'      →  Burstable
```

eviction: kubelet przy presji na węźle · OOMKill: jądro, gdy kontener przekroczy limit

Copyright (C) 2026 Sages sp. z o.o. Kopiowanie, przetwarzanie, rozpowszechnianie tych materiałów w całości lub w części jest zabronione.

sages

Quota liczy sumę, LimitRange uzupełnia braki

RESOURCEQUOTA · SUMA W NAMESPACE

```
apiVersion: v1
kind: ResourceQuota
metadata:
  name: kursant
spec:
  hard:
    requests.cpu: "1"
    requests.memory: 2Gi
    limits.cpu: "3"
    limits.memory: 4Gi
    persistentvolumeclaims: "6"
    requests.storage: 30Gi
```

LIMITRANGE · KAŻDY KONTENER

```
apiVersion: v1
kind: LimitRange
metadata:
  name: kursant-domyslne
spec:
  limits:
  - type: Container
    defaultRequest: {cpu: 50m, memory: 64Mi}
    default: {cpu: 250m, memory: 256Mi}
    max: {cpu: "2", memory: 2Gi}
```

quota na requests bez LimitRange: pod bez resources odpada z „failed quota: must specify…”

Copyright (C) 2026 Sages sp. z o.o. Kopiowanie, przetwarzanie, rozpowszechnianie tych materiałów w całości lub w części jest zabronione.

sages

PriorityClass: ważniejszy pod wypycha mniej ważny

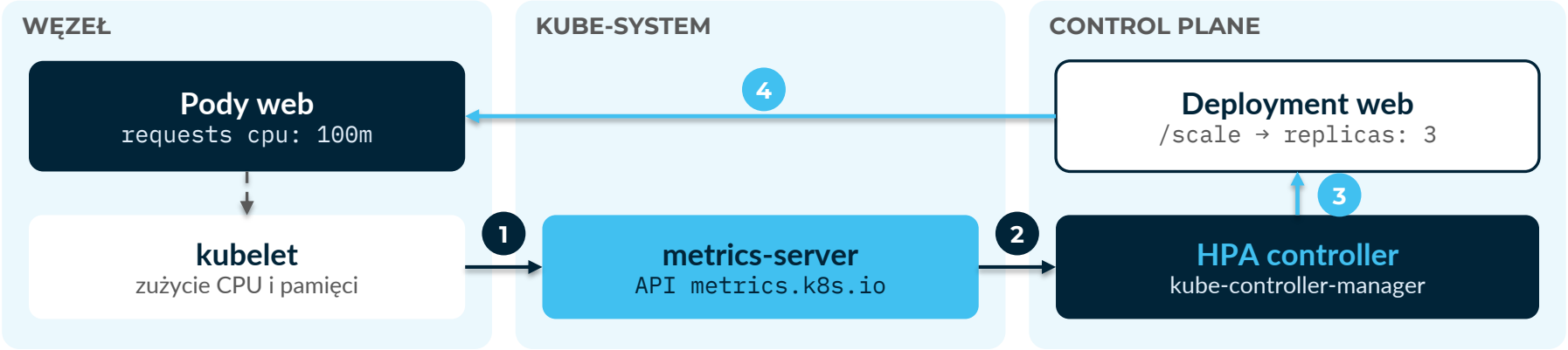
```yaml
apiVersion: scheduling.k8s.io/v1
kind: PriorityClass
metadata:
  name: prod-high
value: 100000
preemptionPolicy: PreemptLowerPriority
globalDefault: false
---
# w podzie:
spec:
  priorityClassName: prod-high
```

1. pod prod-high: Pending, brak miejsca
2. scheduler szuka podów z niższym priorytetem
3. ofiary dostają SIGTERM i czas na zamknięcie
4. pod prod-high wchodzi na zwolnione miejsce

preemptionPolicy: Never = pierwszy w kolejce, ale nikogo nie wypycha

Copyright (C) 2026 Sages sp. z o.o. Kopiowanie, przetwarzanie, rozpowszechnianie tych materiałów w całości lub w części jest zabronione.

HPA co 15 sekund liczy repliki od nowa



desired = ceil(current × aktualne / cel)
2 pody × 90% / 60% = 3 pody

w górę: od razu
w dół: po 300 s stabilizacji · tolerancja 10%

Utilization liczy się od requests: bez requests HPA nie ma od czego liczyć procentów

Copyright (C) 2026 Sages sp. z o.o. Kopiowanie, przetwarzanie, rozpowszechnianie tych materiałów w całości lub w części jest zabronione.

sages

autoscaling/v2: cel, granice i tempo

```
spec:                       # HPA web, autoscaling/v2
  scaleTargetRef:
    apiVersion: apps/v1
    kind: Deployment
    name: web
  minReplicas: 2
  maxReplicas: 10
  metrics:
  - type: Resource
    resource:
      name: cpu
      target: {type: Utilization, averageUtilization: 60}
  behavior:
    scaleDown:
      stabilizationWindowSeconds: 300
      policies: [{type: Pods, value: 1, periodSeconds: 60
}]
```

minReplicas: 2
jedna replika to nie wysoka dostępność

Utilization 60%
zapas na pik, zanim nowe pody wstaną

scaleDown: 1 pod na minutę
łagodne schodzenie po szczycie

więcej metryk = maksimum
HPA bierze najwyższą liczbę replik

Copyright (C) 2026 Sages sp. z o.o. Kopiowanie, przetwarzanie, rozpowszechnianie tych materiałów w całości lub w części jest zabronione.

sages

Poza HPA: VPA, resize w miejscu i KEDA

Mechanizm	Co robi	Warto wiedzieć
HPA · Horizontal Pod Autoscaler	dokłada podów (scale out)	wbudowany, autoscaling/v2
VPA · Vertical Pod Autoscaler	robi pody większe (scale up)	osobny projekt, EKS go nie ma
In-place resize	zmienia resources bez restartu	GA w 1.35, z niego korzysta VPA
KEDA	skaluje od zdarzeń, do zera	pod spodem steruje HPA

VPA + HPA na CPU = pętla: VPA podnosi requests → % CPU spada → HPA zdejmuje repliki

brakuje węzłów? Cluster Autoscaler, Karpenter albo EKS Auto Mode

Copyright (C) 2026 Sages sp. z o.o. Kopiowanie, przetwarzanie, rozpowszechnianie tych materiałów w całości lub w części jest zabronione.

sages

HPA bez requests nie ma od czego liczyć

Objaw	Przyczyna	Co zrobić
TARGETS <unknown>/60%	brak requests.cpu albo metrics-server	dodaj requests, sprawdź kubectl top pod
repliki skaczą 2 ↔ 4	krótkie piki, za niski cel	stabilizationWindowSeconds, cel wyżej
w górę na pamięci, w dół nigdy	pamięć nie spada po ruchu	skaluj na CPU albo metryce ruchu
kubectl scale nic nie daje	HPA nadpisuje replicas	zmień minReplicas w HPA

kubectl describe hpa: Conditions i Events mówią, dlaczego HPA stoi

Copyright (C) 2026 Sages sp. z o.o. Kopiowanie, przetwarzanie, rozpowszechnianie tych materiałów w całości lub w części jest zabronione.

sages

▶ DEMO · OOMKilled i HPA pod obciążeniem

Pamięć zabija, CPU dokłada repliki

Cel: pod z limitem 64Mi zjada pamięć, potem HPA pod ruchem z generatora.

OBSERWUJCIE

1. OOMKilled, exit 137
2. TARGETS ponad 50%
3. REPLICAS 1 → 5, w dół po 5 min

Copyright (C) 2026 Sages sp. z o.o. Kopiowanie, przetwarzanie, rozpowszechnianie tych materiałów w całości lub w części jest zabronione.

sages

LAB `labs/lab11-zasoby-quota-hpa/`

Limity, quota i HPA u siebie

1. QoS: Guaranteed i Burstable, pod bez resources
2. OOMKilled: pod z limitem 64Mi
3. quota: scale do 6 → FailedCreate w ReplicaSecie
4. HPA na CPU i obciążenie; chętni: LimitRange na kind

SUKCES pod bez resources ma wartości z LimitRange, HPA skaluje powyżej 1 i z powrotem

Copyright (C) 2026 Sages sp. z o.o. Kopiowanie, przetwarzanie, rozpowszechnianie tych materiałów w całości lub w części jest zabronione.

sages

Ile CPU zużywa Wasz serwis, a ile ma w requests?

Teraz: ToDos · dzień 3, potem planowanie podów

Mongo na dysku, zasoby i HPA na Waszej aplikacji

Copyright (C) 2026 Sages sp. z o.o. Kopiowanie, przetwarzanie, rozpowszechnianie tych materiałów w całości lub w części jest zabronione.

sages

LAB `labs/todos-dzien3/`

Twój ToDos z dyskiem i autoskalowaniem

1. Mongo: StatefulSet z PVC 1Gi i headless Service
2. resources wszędzie, Mongo w klasie Guaranteed
3. HPA dla api: 2–4 repliki, test obciążeniem
4. (*) CronJob z mongodump na osobny PVC

SUKCES zadanie przeżywa kubectl delete pod mongo-0, a HPA pokazuje % CPU i dokłada repliki

Copyright (C) 2026 Sages sp. z o.o. Kopiowanie, przetwarzanie, rozpowszechnianie tych materiałów w całości lub w części jest zabronione.

sages

DZIEŃ 3 · TEMAT 4

4 Planowanie podów i utrzymanie

Po tym temacie decydujesz, na którym węźle ląduje pod, i wyłączasz węzeł bez przerwy w działaniu aplikacji.

Copyright (C) 2026 Sages sp. z o.o. Kopiowanie, przetwarzanie, rozpowszechnianie tych materiałów w całości lub w części jest zabronione.

sages

Scheduler filtruje, ocenia i przypisuje węzeł

Pod Pending · bez nodeName → Filtr: zasoby, taints, selektory → Ocena: preferencje, rozłożenie → Bind: nodeName = węzeł

nic nie przeszło filtra

pod zostaje Pending, event FailedScheduling:

```
0/3 nodes are available: 1 node(s) had untolerated taint…
```

Ty podpowiadasz

nodeSelector i nodeAffinity: gdzie wolno

pod(Anti)Affinity i spread: obok kogo

taints: gdzie nie wolno bez przepustki

…IgnoredDuringExecution: zasady działają przy planowaniu, działających podów nie przesuwają

Copyright (C) 2026 Sages sp. z o.o. Kopiowanie, przetwarzanie, rozpowszechnianie tych materiałów w całości lub w części jest zabronione.

sages

nodeSelector i nodeAffinity: ten sam cel, inna siła

NODESELECTOR

```
nodeSelector:
  disktype: ssd
```

=

TO SAMO W NODEAFFINITY

```
affinity:
  nodeAffinity:
    requiredDuringSchedulingIgnoredDuringExecution:
      nodeSelectorTerms:
      - matchExpressions:
        - {key: disktype, operator: In, values: [ssd]}
```

	nodeSelector	nodeAffinity
Warunek	tylko „etykieta = wartość”	In, NotIn, Exists, DoesNotExist, Gt, Lt
Siła	tylko musi	musi (required) albo woli (preferred, waga)
Kilka warunków	wszystkie naraz (I)	I w termie, LUB między termami
Brak węzła	Pending	required: Pending · preferred: gdziekolwiek

Copyright (C) 2026 Sages sp. z o.o. Kopiowanie, przetwarzanie, rozpowszechnianie tych materiałów w całości lub w części jest zabronione.

sages

nodeAffinity czytasz od środka: wyrażenie, term, lista

SPEC.AFFINITY.NODEAFFINITY

```
requiredDuringSchedulingIgnoredDuringExecution:  # 1
  nodeSelectorTerms:                             # 2 LUB
  - matchExpressions:                            # 3 I
    - key: disktype                              # 4
      operator: In
      values: [ssd, nvme]
    - key: topology.kubernetes.io/zone
      operator: NotIn
      values: [eu-central-1c]
  -
 matchExpressions:                           # 3 drugi term
    - key: gpu
      operator: Exists
```

1 · required...
musi przy planowaniu; działających podów nie przesuwa

2 · nodeSelectorTerms
lista termów: wystarczy jeden (LUB)

3 · matchExpressions
w jednym termie wszystkie naraz (I)

4 · key · operator · values
etykieta węzła, porównanie, wartości

(ssd/nvme I nie strefa c) LUB gpu

Copyright (C) 2026 Sages sp. z o.o. Kopiowanie, przetwarzanie, rozpowszechnianie tych materiałów w całości lub w części jest zabronione.

sages

Operator mówi, jak porównać etykietę

Operator	values	Węzeł pasuje, gdy...	Przykład
`In`	lista	wartość etykiety jest na liście	`disktype In [ssd, nvme]`
`NotIn`	lista	wartości nie ma na liście	`zone NotIn [eu-central-1c]`
`Exists`	brak	ma etykietę, wartość obojętna	`gpu Exists`
`DoesNotExist`	brak	nie ma tej etykiety	`spot DoesNotExist`
`Gt · Lt`	jedna liczba	wartość większa / mniejsza	`cpu-count Gt ["4"]`

NotIn i DoesNotExist przepuszczają też węzły bez tej etykiety

podAffinity i podAntiAffinity: tylko pierwsze cztery, Gt i Lt są wyłącznie dla węzłów

Copyright (C) 2026 Sages sp. z o.o. Kopiowanie, przetwarzanie, rozpowszechnianie tych materiałów w całości lub w części jest zabronione.

sages

Repliki osobno: anti-affinity albo spread

PODANTIAFFINITY · „NIE OBOK SIEBIE”

```
affinity:
  podAntiAffinity:
    preferredDuringSchedulingIgnoredDuringExecution:
    - weight: 100
      podAffinityTerm:
        labelSelector:
          matchLabels: {app: web}
        topologyKey: kubernetes.io/hostname
```

TOPOLOGYSPREAD · „RÓWNO”

```
topologySpreadConstraints:
- maxSkew: 1
  topologyKey: kubernetes.io/hostname
  whenUnsatisfiable: ScheduleAnyway
  labelSelector:
    matchLabels: {app: web}
```

nodeAffinity	podAffinity	podAntiAffinity
etykiety węzła · na jakim węźle?	inne pody · obok kogo?	inne pody · z dala od kogo?

topologyKey: hostname = węzeł · topology.kubernetes.io/zone = strefa

Copyright (C) 2026 Sages sp. z o.o. Kopiowanie, przetwarzanie, rozpowszechnianie tych materiałów w całości lub w części jest zabronione.

Taint odpycha pody, toleracja to przepustka

WĘZEŁ

```
kubectl taint nodes szkolenie-worker2 dedicated=gpu:NoSchedule     # założenie
kubectl taint nodes szkolenie-worker2 dedicated=gpu:NoSchedule-    # zdjęcie
```

Efekt	Co robi
NoSchedule	nowe pody bez toleracji nie wejdą
PreferNoSchedule	scheduler unika, ale nie zabrania
NoExecute	wyrzuca też działające pody

POD

```
tolerations:
- key: dedicated
  operator: Equal
  value: gpu
  effect: NoSchedule
```

toleracja pozwala, nie przyciąga: węzeł tylko dla GPU = taint + toleracja + nodeAffinity

Copyright (C) 2026 Sages sp. z o.o. Kopiowanie, przetwarzanie, rozpowszechnianie tych materiałów w całości lub w części jest zabronione.

sages

Po co tainty: węzeł broni się sam

Sytuacja	Taint na węźle	Toleracja ma...
drogie węzły: GPU, dużo RAM	`dedicated=gpu:NoSchedule`	pody GPU (+ nodeAffinity)
węzły jednego zespołu	`team=data:NoSchedule`	pody tego zespołu
control plane	`…/control-plane:NoSchedule`	komponenty systemowe
węzeł nie odpowiada	`…/unreachable:NoExecute`	każdy pod, na 300 s
węzły spot na EKS	`capacity=spot:NoSchedule`	aplikacje odporne na wyłączenie

Taint działa na wszystkie pody naraz, affinity tylko na pod, który ją ma

Copyright (C) 2026 Sages sp. z o.o. Kopiowanie, przetwarzanie, rozpowszechnianie tych materiałów w całości lub w części jest zabronione.

sages

Kiedy co: do czego jest, czego nie robi

Mechanizm	Do czego	Czego NIE robi	Siła
nodeSelector	węzeł z daną etykietą	nie zna LUB ani „wolę”	twardo
nodeAffinity	gdzie wolno, gdzie lepiej	nie odpycha innych podów	oba
podAffinity	obok kogo (cache)	nie przyciąga tamtych podów	oba
podAntiAffinity	repliki osobno	nie rozkłada równo	oba
topologySpread	równo po węzłach, strefach	nie wybiera, które węzły	maxSkew
taint + toleracja	węzeł tylko dla wybranych	toleracja nie kieruje na węzeł	twardo

Działających podów nie przesuwa żaden, wyjątek: taint NoExecute

Copyright (C) 2026 Sages sp. z o.o. Kopiowanie, przetwarzanie, rozpowszechnianie tych materiałów w całości lub w części jest zabronione.

sages

Serwis węzła: cordon, drain, uncordon

cordon: SchedulingDisabled → drain: eviction z PDB → prace, upgrade, wymiana → uncordon

PODDISRUPTIONBUDGET

```
apiVersion: policy/v1
kind: PodDisruptionBudget
metadata:
  name: web-pdb
spec:
  minAvailable: 2
  selector:
    matchLabels: {app: web}
```

KOMENDY

```
kubectl cordon <węzeł>
kubectl drain <węzeł> --ignore-daemonsets \
  --delete-emptydir-data
kubectl uncordon <węzeł>
# EKS: upgrade grupy węzłów robi to sam
aws eks update-nodegroup-version \
  --cluster-name k8s-sages --nodegroup-name <ng>
```

drain czeka na PDB? To PDB działa · EKS: najpierw nowy węzeł, potem drain starego

Copyright (C) 2026 Sages sp. z o.o. Kopiowanie, przetwarzanie, rozpowszechnianie tych materiałów w całości lub w części jest zabronione.

sages

PDB pilnuje tylko dobrowolnych zakłóceń

CHRONI · PYTAJĄ PRZEZ EVICTION API

kubectl drain

upgrade i wymiana grupy węzłów EKS

Cluster Autoscaler, Karpenter zdejmują węzeł

NIE CHRONI · NIKT NIE PYTA

awaria węzła, OOMKill

kubectl delete pod

rolling update: maxUnavailable i maxSurge

SPRAWDZENIE

```
kubectl get pdb   →   web-pdb   MIN AVAILABLE 2   ALLOWED DISRUPTIONS 1   (3 pody Ready)
```

ALLOWED DISRUPTIONS = zdrowe pody – minAvailable · 0 = drain i upgrade czekają

Copyright (C) 2026 Sages sp. z o.o. Kopiowanie, przetwarzanie, rozpowszechnianie tych materiałów w całości lub w części jest zabronione.

sages

Za ostry PDB blokuje drain na zawsze

Ustawienie	Skutek	Lepiej
minAvailable = replicas	drain wisi: „would violate … budget”	maxUnavailable: 1
1 replika + minAvailable: 1	ALLOWED DISRUPTIONS zawsze 0	min. 2 repliki
pody nie-Ready pod PDB	zepsuta aplikacja blokuje drain	unhealthyPodEvictionPolicy: AlwaysAllow
required anti-affinity, 5 replik	na 3 węzłach 2 pody Pending	preferred albo spread
drain bez --ignore-daemonsets	cannot delete DaemonSet-managed Pods	dodaj flagę

każda twarda reguła ma koszt: Pending albo wiszący drain w najgorszym momencie

Copyright (C) 2026 Sages sp. z o.o. Kopiowanie, przetwarzanie, rozpowszechnianie tych materiałów w całości lub w części jest zabronione.

sages

Który węzeł w Waszym klastrze wyłączylibyście dziś bez strachu?

Na koniec: podsumowanie dnia 3

cztery tematy w czterech zdaniach i co jutro

Copyright (C) 2026 Sages sp. z o.o. Kopiowanie, przetwarzanie, rozpowszechnianie tych materiałów w całości lub w części jest zabronione.

sages

Dzień 3: cztery tematy, cztery zdania

Temat	Zapamiętaj
Storage i StatefulSet	PVC zamawia, StorageClass tworzy, StatefulSet daje imię i dysk
DaemonSet, Job, CronJob	węzły, sukcesy, zegar · sprzątanie ustawiasz sam
Zasoby, QoS, HPA	requests rezerwują, limits przycinają, HPA liczy od requests
Planowanie i utrzymanie	affinity przyciąga, taint odpycha, PDB chroni przy drain

Jutro, dzień 4: RBAC i ServiceAccount · Pod Security · NetworkPolicy i DNS · CRD

Copyright (C) 2026 Sages sp. z o.o. Kopiowanie, przetwarzanie, rozpowszechnianie tych materiałów w całości lub w części jest zabronione.